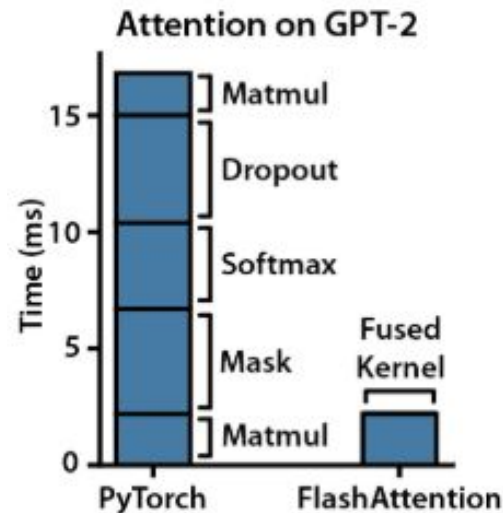


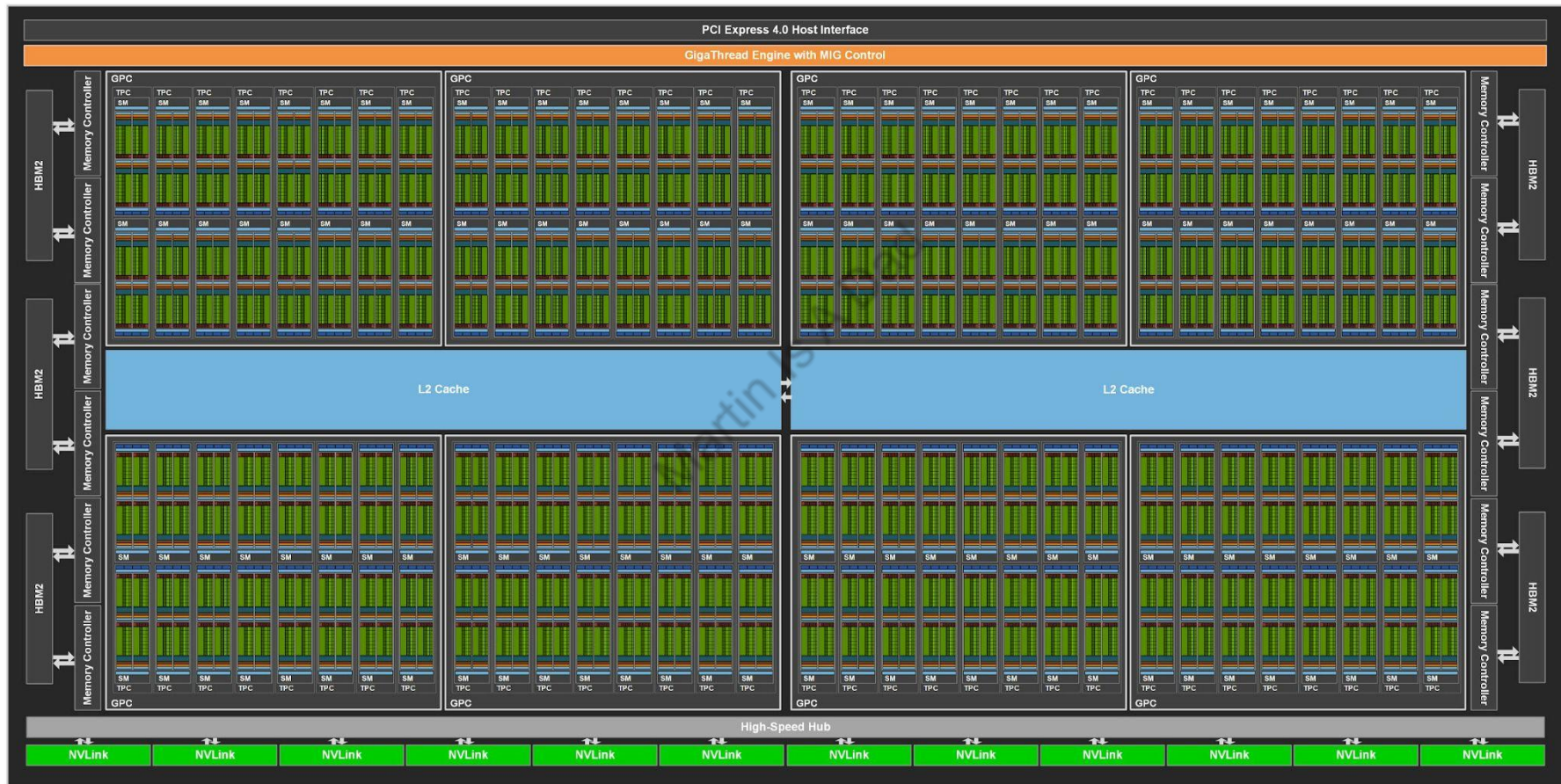
Motivation Of FlashAttention V2

- As Transformer based LLMs needs to work on longer sequence, attention layer becomes the main bottleneck of scaling up. This is the main motivation for FlashAttention V1.
- FlashAttention V1 shows **2X4 runtime speedup** compared to optimized standard Attention implementations. However, this is not nearly as fast as optimized Matmul (or GEMM, General Matrix Multiplications), only 25%~40% of the theoretical maximum FLOPs.
- FlashAttention V2 reach another **2X runtime speedup** compared to FlashAttention V1, reaching 50~73% of the theoretical maximum FLOPs. This is achieved by:
 - Reduce non-Matmul operations
 - Parallelize on sequence length dimension to optimize utilization
 - Better execution ordering to reduce I/O
 - Better partitioning the work between warps to reduce I/O



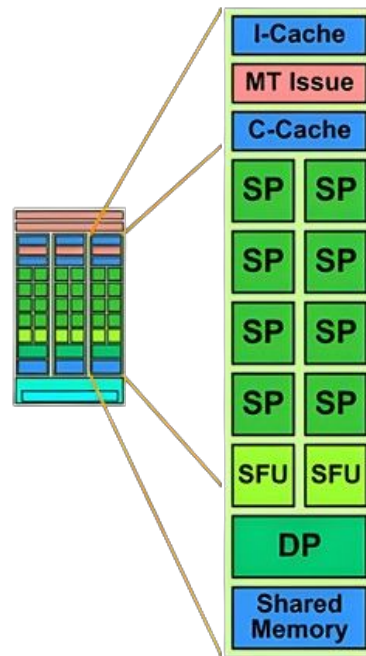
[\[arxiv 2205.14135\]](https://arxiv.org/abs/2205.14135) Runtime comparison of FlashAttention and Standard Attention

GPU High Level



GPU Streaming Multiprocessors (SMs)

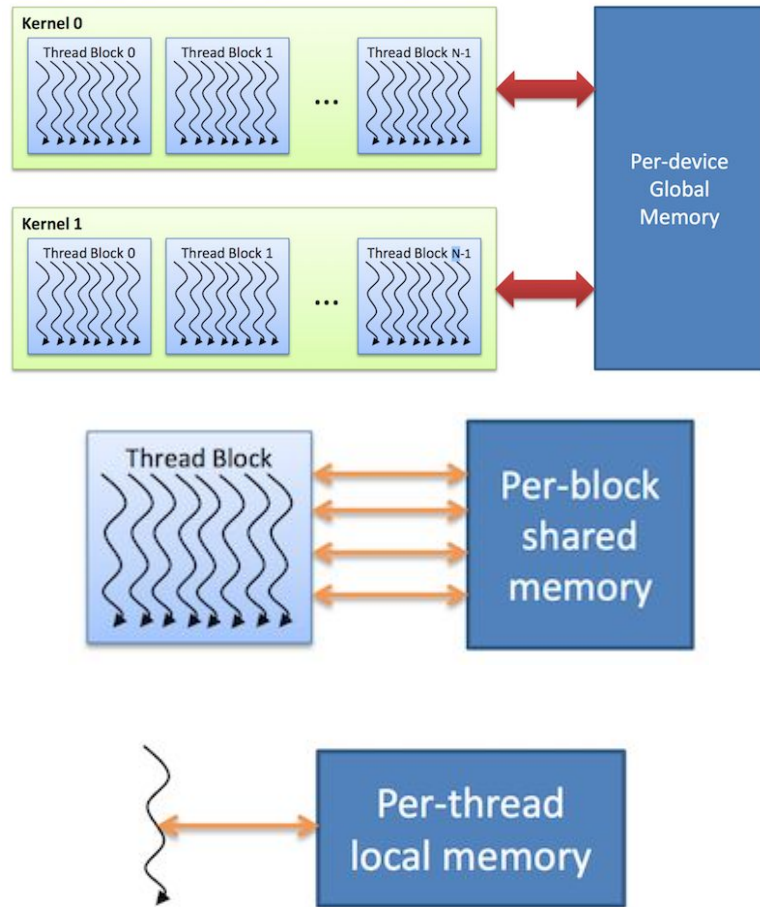
- **Streaming Multiprocessors (SMs):** The fundamental processing unit within a modern GPU. A GPU is composed of many SMs. Each SM is like a mini-processor capable of executing many threads concurrently. SMs usually contains:
 - CUDA Cores (or SP, Stream Processors): fundamental execution units for general purpose
 - Special units and Tensor Cores: specialized hardware for modern AI
 - Registers, L1 Cache, Shared Memory etc
- **Threads:** These are the smallest units of execution. When you run a computation on a GPU (like a part of an LLM's neural network layer), it's broken down into thousands or millions of threads.
- **Thread Blocks:** Threads are organized into blocks. Threads within the same block can cooperate by sharing data through a fast on-chip memory (shared memory) and synchronizing their execution.



Nvidia Streaming Multiprocessor

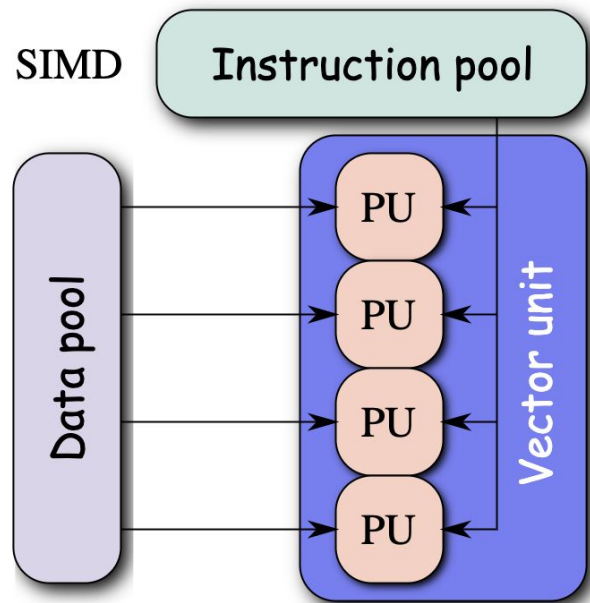
GPU Threads & Memory

- **Global Memory (VRAM / HBM):** It is accessible by all threads in all thread blocks and the host CPU. While it has high bandwidth, it also has the highest latency. **HBM** is a type of technology used to implement VRAM.
- **Shared Memory:** A programmer-managed space that is shared by all threads within a thread block. It provides a high-speed way for threads in the same block to communicate and share data, significantly reducing the need to access the much slower global memory.
- **Local Memory:** Each thread has its own private local memory, it only exists for the lifetime of the thread. This is generally handled automatically by the compiler, say registers & spill over on DRAM etc



GPU Warp & SIMD

- **Warp** is a Group of threads. Within a thread block, threads are further grouped into warps. A warp is typically a set of 32 threads in NVIDIA chip's architecture.
- **SIMD** Execution (Single Instruction, Multiple Data): The key characteristic of a warp is that all threads within it execute the same instruction at the same time, but on different data.
 - If one thread in a warp has to wait (say for I/O), often the entire warp stalls, although modern GPUs have schedulers that can switch to other ready warps.
 - SM's warp scheduler selects which warp gets to execute its next instruction.

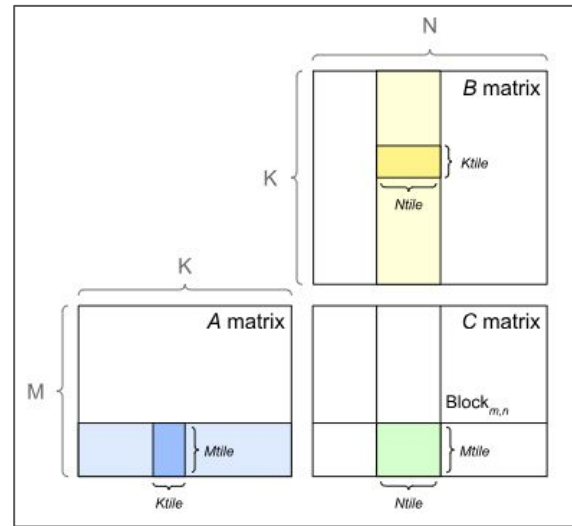


From [Wikipedia SIMD](#)

GPU GEMMs Optimization

NVIDIA GPUs optimize GEMM through tiling, shared memory access optimization, Tensor Core utilization, and the use of libraries like cuBLAS and CUTLASS. These strategies combine to enhance data reuse, minimize memory bandwidth, and leverage the GPU's parallel processing capabilities for high-performance matrix multiplication.

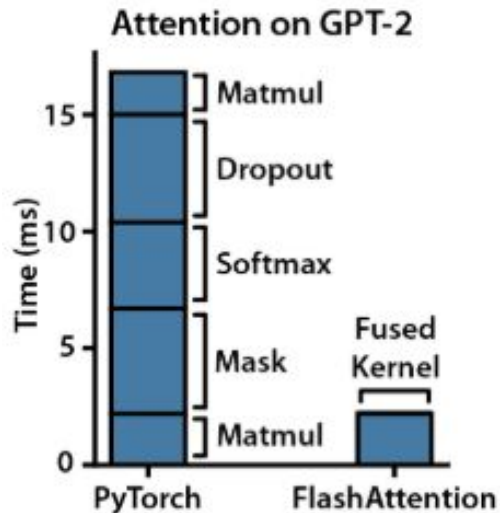
- **Tiling:** Breaking down large matrices into smaller tiles that fit into the GPU's shared memory with better I/O.
- **Shared Memory Access Optimization:** Loading data from global memory to shared memory (with better I/O) in batches and perform computations there
- **Tensor Core:** specialized hardware with high-performance matrix multiplication, especially for lower precision (e.g., FP4, FP8).
- **cuBLAS and CUTLASS library:** NVIDIA's cuBLAS library provides optimized GEMM kernels for various matrix sizes and precisions, including FP8 and FP4.



[\[docs.nvidia.com\]](https://docs.nvidia.com/cuda/cublas/index.html) [NVIDIA GPU GEMMs Tiling](https://docs.nvidia.com/cuda/cublas/index.html)

GPU GEMMs Optimization

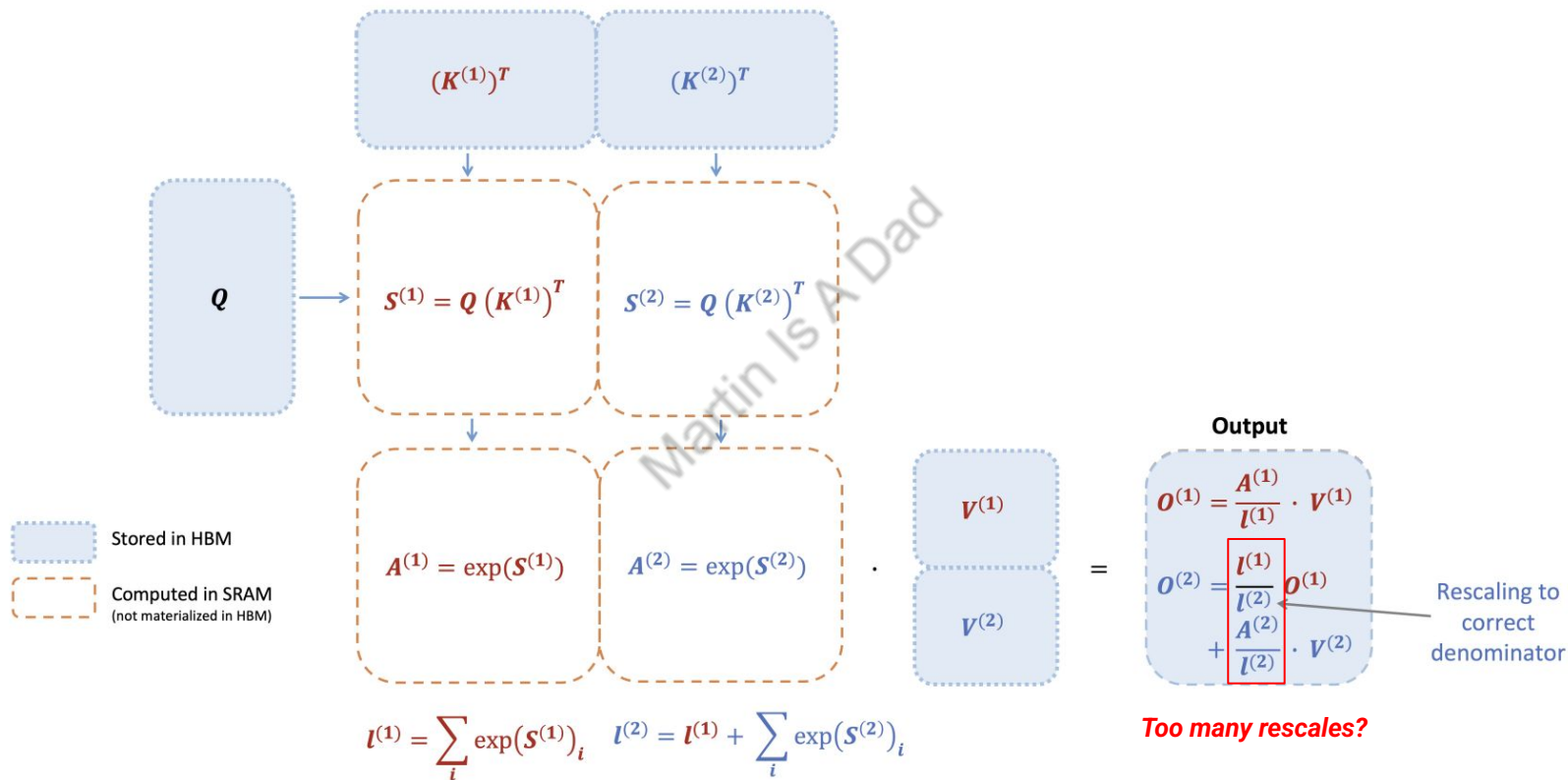
- Using NVIDIA's A100 GPU as example, it's max theoretical throughput is **312** TFLOPs/s of FP16/BF16 matmul, but only **19.5** TFLOPs/s of non-matmul FP32.
- This means each non-matmul FLOP is **16×** more expensive than a matmul FLOP. To maintain high throughput (e.g., more than 50% of the maximum theoretical TFLOPs/s), we want to spend as much time on matmul FLOPs as possible, or reduce non-matmul FLOPs as much as possible



NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS
(SXM4 AND PCIE FORM FACTORS)

	A100 40GB PCIe	A100 80GB PCIe	A100 40GB SXM	A100 80GB SXM
FP64	9.7 TFLOPS			
FP64 Tensor Core	19.5 TFLOPS			
FP32	19.5 TFLOPS			
Tensor Float 32 (TF32)	156 TFLOPS 312 TFLOPS*			
BFLOAT16 Tensor Core	312 TFLOPS 624 TFLOPS*			
FP16 Tensor Core	312 TFLOPS 624 TFLOPS*			
INT8 Tensor Core	624 TOPS 1248 TOPS*			

Recap Of FlashAttention V1



Reduce Non-Matmul FLOPs: Forward Pass

- We do not have to rescale both terms of the output update by $\text{diag}(\ell^{(2)})^{-1}$ and save those FLOPs

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)} / \ell^{(2)})^{-1} \mathbf{O}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - \mathbf{m}^{(2)}} \mathbf{V}^{(2)}$$

we can instead maintain an "unscaled" version of $\mathbf{O}^{(2)}$ and keep around $\ell^{(2)}$.

$$\tilde{\mathbf{O}}^{(2)} = \text{diag}(\ell^{(1)})^{-1} \mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)} - \mathbf{m}^{(2)}} \mathbf{V}^{(2)}$$

At the every end of the loop do we scale the final $\tilde{\mathbf{O}}^{(\text{last})}$ by $\text{diag}(\ell^{(\text{last})})^{-1}$ to get the right output.

- We do not have to save both the max $m^{(i)}$ and the sum of exponentials $\ell^{(i)}$ for the backward pass. We only need to store the logsumexp $L^{(i)} = m^{(i)} + \log(\ell^{(i)})$

$$\mathbf{m}^{(1)} = \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r}$$

$$\ell^{(1)} = \text{rowsum}(e^{\mathbf{S}^{(1)} - \mathbf{m}^{(1)}}) \in \mathbb{R}^{B_r}$$

$$\tilde{\mathbf{P}}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - \mathbf{m}^{(1)}} \in \mathbb{R}^{B_r \times B_c}$$

$$\mathbf{O}^{(1)} = \tilde{\mathbf{P}}^{(1)} \mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - \mathbf{m}^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d}$$

$$\mathbf{m}^{(2)} = \max(\mathbf{m}^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m$$

$$\ell^{(2)} = e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell$$

$$\tilde{\mathbf{P}}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}}$$

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)} / \ell^{(2)})^{-1} \mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)} \mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(1)} - m} \mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m} \mathbf{V}^{(2)} = \mathbf{O}.$$

FlashAttention V1 Forward Pass

$$\mathbf{m}^{(1)} = \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r}$$

$$\ell^{(1)} = \text{rowsum}(e^{\mathbf{S}^{(1)} - \mathbf{m}^{(1)}}) \in \mathbb{R}^{B_r}$$

$$\tilde{\mathbf{O}}^{(1)} = e^{\mathbf{S}^{(1)} - \mathbf{m}^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d}$$

$$\mathbf{m}^{(2)} = \max(\mathbf{m}^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m$$

$$\ell^{(2)} = e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell$$

$$\tilde{\mathbf{P}}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}}$$

$$\tilde{\mathbf{O}}^{(2)} = \text{diag}(e^{m^{(1)} - m^{(2)}}) \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{\mathbf{S}^{(1)} - m} \mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)} - m} \mathbf{V}^{(2)}$$

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.$$

FlashAttention V2 Forward Pass

Reduce Non-Matmul FLOPs: Forward Pass

Algorithm 1 FLASHATTENTION-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
 - 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$, $\ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$, $m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$
(pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
 - 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
 - 11: **end for**
 - 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$. Scale at end of loop
 - 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
 - 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
 - 15: Write L_i to HBM as the i -th block of L . Store $L^{(j)}$ directly instead of $m^{(j)}$ and $\ell^{(j)}$
 - 16: **end for**
 - 17: Return the output $\mathbf{O}$ and the logsumexp L .
-

Reduce Non-Matmul FLOPs: Backward Pass

Algorithm 2 FLASHATTENTION-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
 - 3: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
 - 4: Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do** Compute D ahead as a whole, more efficient
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
 - 8: **for** $1 \leq i \leq T_r$ **do**
 - 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
 - 10: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 11: On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - L_i) \in \mathbb{R}^{B_r \times B_c}$. Use $L^{(j)}$ instead of $m^{(j)}$ and $\ell^{(j)}$
 - 12: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
 - 13: On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
 - 14: On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
 - 15: Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
 - 16: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
 - 17: **end for**
 - 18: Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
 - 19: **end for**
 - 20: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-

Better Parallelization (Among Thread Blocks)

- FlashAttention V1 parallelizes over batch size and number of heads. So there are overall (*batch size $\times$ number of heads*) thread blocks. Each thread block is scheduled to run on a streaming multiprocessor (SM). Using Nvidia's A100 GPU for example, there are 108 of SMs. Computation scheduled on different SMs are executed parallelly and require no synchronization between them.
 - Scheduling is efficient when (*batch size $\times$ number of heads*) is large (say ≥ 80), to effectively use almost all of the compute resources on the GPU.
 - However, In the case of long sequences (which usually means small batch sizes or small number of heads due to GPU memory limit), (*batch size $\times$ number of heads*) is small and GPU is not utilized efficiently.
- To have better GPU parallelism, especially on long sequences, FlashAttention V2 additionally parallelize over the sequence length dimension. This mean there will be (*batch size $\times$ number of heads $\times$ sequence length*) thread blocks needed, and it'd be sufficiently large even when batch size is small.

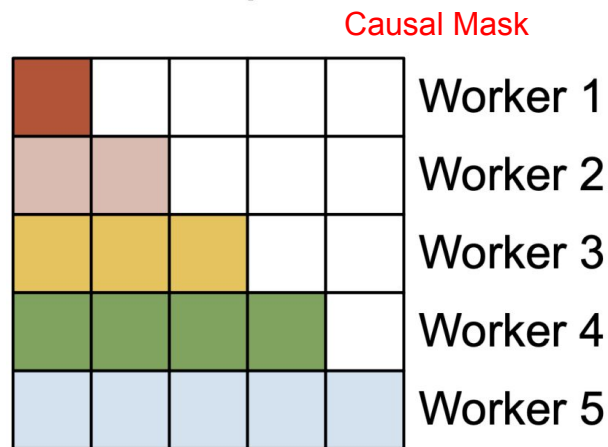
Better Parallelization: Forward Pass

During forward pass, FlashAttention V2 parallelizes by assigning each worker a block of rows from the query matrix. Each worker then computes the attention output for its assigned queries independently.

- Worker 1 takes the first block of queries and computes their attention output by attending to all the key-value pairs.
- Worker 2 simultaneously takes the second block of queries and does the same.
- This continues for all workers, as depicted by the horizontal slicing in the "Forward pass" diagram. The rows represent the queries, and the columns represent the keys/values. Each colored block shows a worker processing its set of queries against a corresponding block of keys.

Since the attention output for each query is independent of the others, there's no need for communication between workers in the forward pass, making it highly efficient.

Forward pass



[\[arxiv 2307.08691\]](#) Forward Pass Attention
Matrix Parallelism

Better Parallelization: Forward Pass

Algorithm 1 FLASHATTENTION-2 forward pass

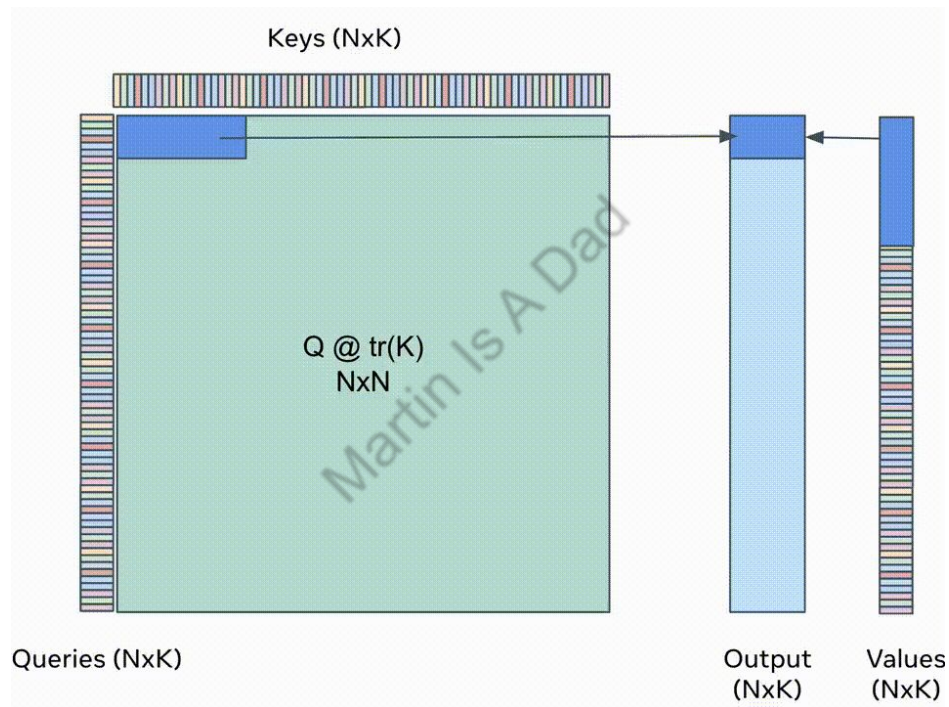
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
 - 5: On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$, $\ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$, $m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 8: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 9: On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
 - 10: On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
 - 11: **end for**
 - 12: On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
 - 13: On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
 - 14: Write $\mathbf{O}_i$ to HBM as the i -th block of $\mathbf{O}$.
 - 15: Write L_i to HBM as the i -th block of L .
 - 16: **end for**
 - 17: Return the output $\mathbf{O}$ and the logsumexp L .
-

Outer loop on Queries

Inner loop on Keys
and Values

Better Parallelization: Forward Pass



From [Francisco Massa @ X.com](#)

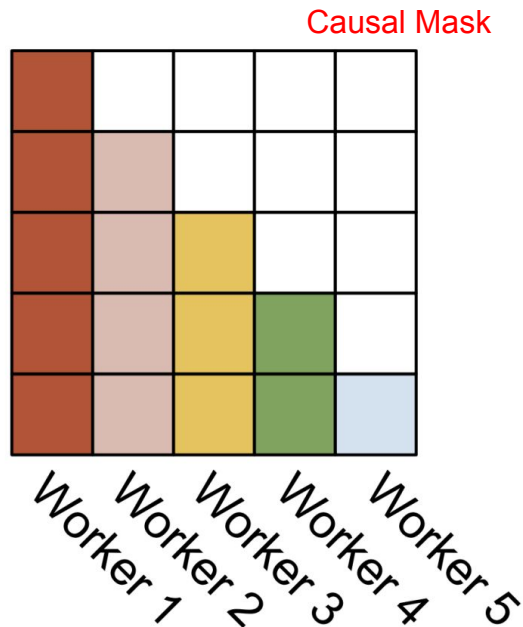
Better Parallelization: Backward Pass

The backward pass involves calculating the gradients of the Query, Key, and Value matrices, which is key for training the model. Here, the parallelization strategy shifts. Instead of splitting the queries, FlashAttention V2 parallelizes over the key/value pairs (same as V1).

- As shown in the "Backward pass" diagram, each worker is now responsible for a block of columns of the attention score matrix. This means each worker calculates the gradient contribution from a subset of the key-value pairs to all the queries.
- Worker 1 processes the first block of key-value pairs and their influence on the gradients of all queries.
- Worker 2 handles the second block, and so on.

Unlike the forward pass, the backward pass requires some communication (atomic operations) between workers to correctly aggregate the gradients for the query matrix. However, this column-wise parallelization is more efficient for the backward pass as it reduces the amount of data that needs to be communicated.

Backward pass



[\[arxiv 2307.08691\]](https://arxiv.org/abs/2307.08691) Backward Pass Attention Matrix Parallelism

Better Parallelization: Backward Pass

Algorithm 2 FLASHATTENTION-2 Backward Pass

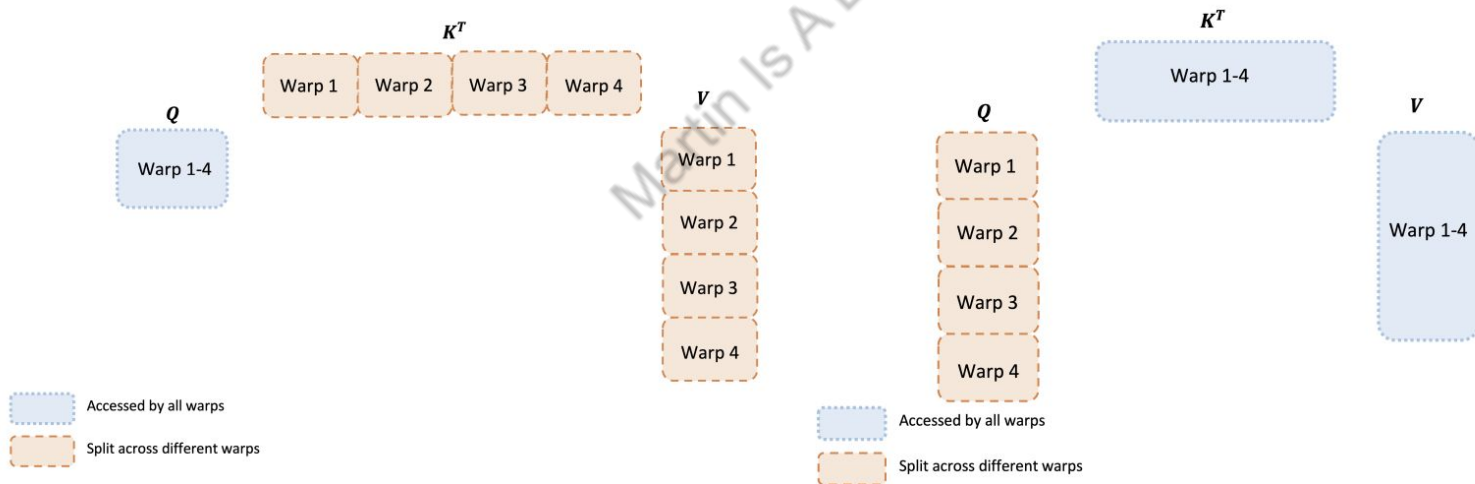
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r .

- 1: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 2: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
 - 3: Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
 - 4: Compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.
 - 8: **for** $1 \leq i \leq T_r$ **do**
 - 9: Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.
 - 10: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 11: On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
 - 12: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
 - 13: On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
 - 14: On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
 - 15: Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.
 - 16: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
 - 17: **end for**
 - 18: Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.
 - 19: **end for**
 - 20: Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.
-

- The only shared computation between different column blocks is in update $\mathbf{dQ}$, where we need to load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip update $\mathbf{dQ}_i$ and write back to HBM.
- We achieve this by scheduling 1 thread block for each column block of the backward pass and use atomic adds to communicate between different thread blocks to update $\mathbf{dQ}$.

Better Warp Partition (Within Single Thread Block)

- Note that the paper's author was using 4 warps per thread block, thus the graph below.
- FlashAttention V1 splits $\mathbf{K}$ and $\mathbf{V}$ across 4 warps while keeping $\mathbf{Q}$ accessible by all warps. Each warp multiplies to get a slice of $\mathbf{QK}^T$, then multiply with a slice of $\mathbf{V}$ and communicate to add up the result. This is inefficient since all warps need to write their intermediate results out to shared memory, synchronize, then add up the intermediate results. These shared memory reads/writes slow down the forward pass.

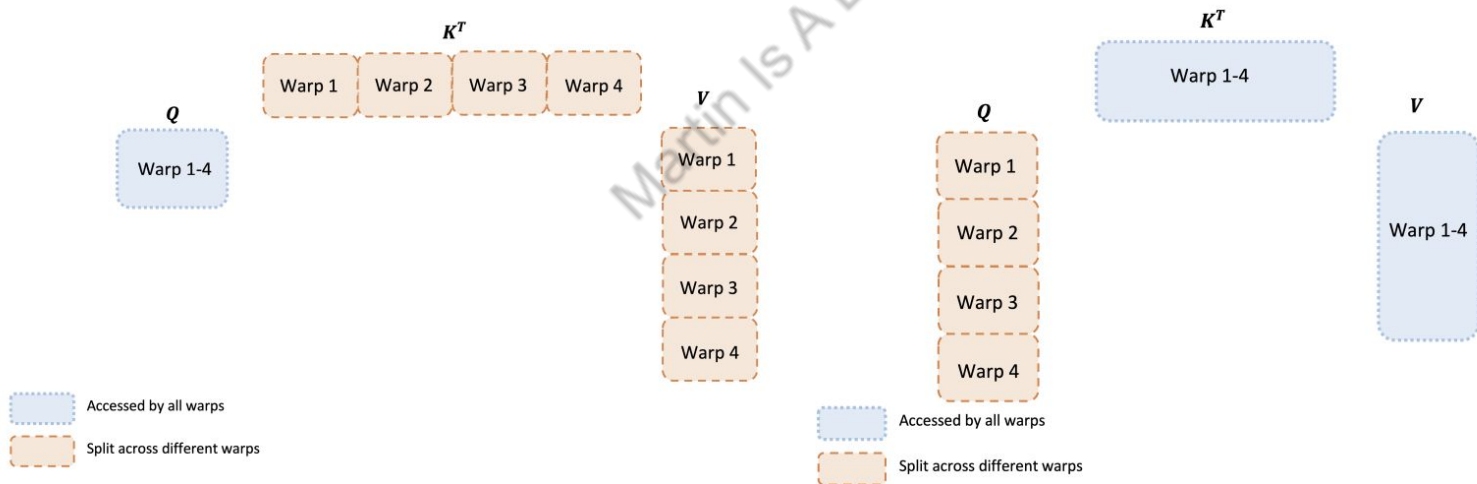


(a) FLASHATTENTION

(b) FLASHATTENTION-2

Better Warp Partition (Within Single Thread Block)

- In forward pass, FlashAttention V2 instead split Q across 4 warps and keep K , V accessible by all warps. After each warp performs matmul to get a slice of QK^T , they just need to multiply with their shared slice of V to get their corresponding slice of the output. There is no need for communication between warps. The reduction in shared memory reads/writes contributes to runtime speedup
- FlashAttention V2's backward pass has similar partition, but it still requires some synchronization between warps.



(a) FLASHATTENTION

(b) FLASHATTENTION-2